

Top-Down Parsing

CS143

Lecture 7

Instructor: Fredrik Kjolstad

Slide design by Prof. Alex Aiken, with modifications

Predictive Top-Down Parsers

- Like recursive-descent but parser can “predict” which production to use
 - By looking at the next few tokens
 - No backtracking
- Predictive parsers accept LL(k) grammars
 - L means “left-to-right” scan of input
 - L means “leftmost derivation”
 - k means “predict based on k tokens of lookahead”
 - In practice, LL(1) is used

Recursive Descent vs. LL(1)

- In recursive-descent,
 - At each step, many choices of production to use
 - Backtracking used to undo bad choices
- In LL(1),
 - At each step, only one choice of production
 - That is
 - When a non-terminal A is leftmost in a derivation
 - And the next input symbol is t
 - There is a unique production $A \rightarrow \alpha$ to use
 - Or no production to use (an error state)
- LL(1) is a recursive descent variant without backtracking

Predictive Parsing and Left Factoring

- Recall the grammar

$$E \rightarrow T + E \mid T$$
$$T \rightarrow \text{int} \mid \text{int} * T \mid (E)$$

- Hard to predict because
 - For T two productions start with int
 - For E it is not clear how to predict
- We need to left-factor the grammar

Left-Factoring Example

- Recall the grammar

$$E \rightarrow T + E \mid T$$

$$T \rightarrow \text{int} * T \mid \text{int} \mid (E)$$

- Factor out common prefixes of productions

$$E \rightarrow T X$$

$$X \rightarrow + E \mid \varepsilon$$

$$T \rightarrow \text{int} Y \mid (E)$$

$$Y \rightarrow * T \mid \varepsilon$$

LL(1) Parsing Table Example

- Left-factored grammar

$$E \rightarrow T X$$

$$X \rightarrow + E \mid \varepsilon$$

$$T \rightarrow \text{int } Y \mid (E)$$

$$Y \rightarrow * T \mid \varepsilon$$

- The LL(1) parsing table: next input token

	int	*	+	(	)	\$
E	$T X$			$T X$		
X			$+ E$		ε	ε
T	$\text{int } Y$			(E)		
Y		$* T$	ε		ε	ε

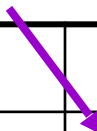
leftmost non-terminal

rhs of production to use

$$\begin{array}{ll}
 E \rightarrow T X & X \rightarrow + E \mid \varepsilon \\
 T \rightarrow \text{int } Y \mid (E) & Y \rightarrow * T \mid \varepsilon
 \end{array}$$

LL(1) Parsing Table Example

- Consider the $[E, \text{int}]$ entry
 - “When current non-terminal is E and next input is int , use production $E \rightarrow T X$ ”
 - This can generate an int in the first position



	int	*	+	(	)	\$
E	TX			TX		
X			+ E		ε	ε
T	int Y			(E)		
Y		* T	ε		ε	ε

$$\begin{array}{ll}
 E \rightarrow T X & X \rightarrow + E \mid \varepsilon \\
 T \rightarrow \text{int } Y \mid (E) & Y \rightarrow * T \mid \varepsilon
 \end{array}$$

LL(1) Parsing Tables. Errors

- Consider the $[Y, +]$ entry
 - “When current non-terminal is Y and current token is $+$, get rid of Y ”
 - Y can be followed by $+$ only if $Y \rightarrow \varepsilon$

	int	*	+	(	)	\$
E	$T X$			$T X$		
X			$+ E$		ε	ε
T	$\text{int } Y$			(E)		
Y		$* T$	ε		ε	ε



$$E \rightarrow T X \quad X \rightarrow + E \mid \varepsilon$$

$$T \rightarrow \text{int } Y \mid (E) \quad Y \rightarrow * T \mid \varepsilon$$

LL(1) Parsing Tables. Errors

- Consider the $[Y, (]$ entry
 - “There is no way to derive a string starting with $($ from non-terminal Y ”
 - Blank entries indicate error situations

	int	*	+	(	)	\$
E	$T X$			$T X$		
X			$+ E$		ε	ε
T	$\text{int } Y$			(E)		
Y		$* T$	ε		ε	ε



Using Parsing Tables

- Method similar to recursive descent, except
 - For the leftmost non-terminal S
 - We look at the next input token a
 - And choose the production shown at $[S,a]$
- A stack records frontier of parse tree
 - Non-terminals that have yet to be expanded
 - Terminals that have yet to be matched against the input
 - Top of stack = leftmost pending terminal or non-terminal
- Reject on reaching error state
- Accept on end of input & empty stack

LL(1) Parsing Algorithm (using the table)

initialize stack = $\langle S \$ \rangle$ and next

repeat

case stack of

$\langle X, \text{rest} \rangle$: if $T[X, *next] = Y_1 \dots Y_n$
then $\text{stack} \leftarrow \langle Y_1 \dots Y_n, \text{rest} \rangle$;
else error ();

$\langle t, \text{rest} \rangle$: if $t == *next++$
then $\text{stack} \leftarrow \langle \text{rest} \rangle$;
else error ();

until $\text{stack} == \langle \rangle$

LL(1) Parsing Algorithm

\$ marks bottom of stack

initialize stack = $\langle S \ \$ \rangle$ and next

repeat

case stack of

$\langle X, \text{rest} \rangle$: if $T[X, *next] = Y_1 \dots Y_n$
then stack $\leftarrow \langle Y_1 \dots Y_n, \text{rest} \rangle$;
else error ();

$\langle t, \text{rest} \rangle$: if $t == *next ++$
then stack $\leftarrow \langle \text{rest} \rangle$;
else error ();

For terminal t on top of stack,
check t matches next input
token.

until stack == $\langle \ \rangle$

For non-terminal X on top of stack,
lookup production

Pop X , push
production rhs
on stack.
Note leftmost
symbol of rhs
is on top of
the stack.

$$\begin{array}{ll}
 E \rightarrow T X & X \rightarrow + E \mid \varepsilon \\
 T \rightarrow \text{int } Y \mid (E) & Y \rightarrow * T \mid \varepsilon
 \end{array}$$

LL(1) Parsing Example

Stack	Input	Action
E \$	int * int \$	T X
T X \$	int * int \$	int Y
int Y X \$	int * int \$	terminal
Y X \$	* int \$	* T
* T X \$	* int \$	terminal
T X \$	int \$	int Y
int Y X \$	int \$	terminal
Y X \$	\$	ε
X \$	\$	ε
\$	\$	ACCEPT

Constructing Parsing Tables: The Intuition

- Consider non-terminal A , production $A \rightarrow \alpha$, and token t

1. Add $T[A, t] = \alpha$

if $A \rightarrow \alpha \rightarrow^* t \beta$

- α can derive a t in the first position
- We say that $t \in \text{First}(\alpha)$

Greek letters denote strings of non-terminals and terminals



2. Add $T[A, t] = \epsilon$

if $A \rightarrow \alpha \rightarrow^* \epsilon$ and $S \rightarrow^* \gamma A t \delta$

- Useful if stack has A , input is t , and A cannot derive t
- In this case only option is to get rid of A (by deriving ϵ)
 - Can work only if t can follow A in at least one derivation
- We say $t \in \text{Follow}(A)$

Computing First Sets

Definition

$$\text{First}(X) = \{ t \mid X \rightarrow^* t\alpha \} \cup \{ \varepsilon \mid X \rightarrow^* \varepsilon \}$$

Algorithm sketch:

1. $\text{First}(t) = \{ t \}$
2. $\varepsilon \in \text{First}(X)$
 - if $X \rightarrow \varepsilon$ or
 - if $X \rightarrow A_1 \dots A_n$ and $\varepsilon \in \text{First}(A_i)$ for all $1 \leq i \leq n$
3. $\text{First}(\alpha) \subseteq \text{First}(X)$
 - if $X \rightarrow \alpha$ or
 - if $X \rightarrow A_1 \dots A_n \alpha$ and $\varepsilon \in \text{First}(A_i)$ for all $1 \leq i \leq n$

First Sets: Example

1. $\text{First}(t) = \{ t \}$
2. $\varepsilon \in \text{First}(X)$
 - if $X \rightarrow \varepsilon$ or
 - if $X \rightarrow A_1 \dots A_n$ and $\varepsilon \in \text{First}(A_i)$ for all $1 \leq i \leq n$
3. $\text{First}(\alpha) \subseteq \text{First}(X)$
 - if $X \rightarrow \alpha$ or
 - if $X \rightarrow A_1 \dots A_n \alpha$ and $\varepsilon \in \text{First}(A_i)$ for all $1 \leq i \leq n$

$E \rightarrow T X$ $X \rightarrow + E \mid \varepsilon$

$T \rightarrow \text{int } Y \mid (E)$ $Y \rightarrow * T \mid \varepsilon$

$\text{First}(E) =$

$\text{First}(X) =$

$\text{First}(T) =$

$\text{First}(Y) =$

First Sets: Example

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow \text{int } Y \mid (E)$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- First sets

$$\text{First}(()) = \{ (\}$$

$$\text{First}()) = \{) \}$$

$$\text{First}(\text{int}) = \{ \text{int} \}$$

$$\text{First}(+) = \{ + \}$$

$$\text{First}(*) = \{ * \}$$

$$\text{First}(T) = \{ \text{int}, (\}$$

$$\text{First}(E) = \{ \text{int}, (\}$$

$$\text{First}(X) = \{ +, \varepsilon \}$$

$$\text{First}(Y) = \{ *, \varepsilon \}$$

Computing Follow Sets

- Definition:

$$\text{Follow}(X) = \{ t \mid S \rightarrow^* \beta X t \delta \}$$

- Intuition

- If $X \rightarrow A B$ then $\text{First}(B) \subseteq \text{Follow}(A)$ and
 $\text{Follow}(X) \subseteq \text{Follow}(B)$
 - if $B \rightarrow^* \varepsilon$ then $\text{Follow}(X) \subseteq \text{Follow}(A)$
- If S is the start symbol then $\$ \in \text{Follow}(S)$

Computing Follow Sets (Cont.)

Algorithm sketch:

1. $\$ \in \text{Follow}(S)$
2. For each production $A \rightarrow \alpha X \beta$
 - $\text{First}(\beta) - \{\epsilon\} \subseteq \text{Follow}(X)$
3. For each production $A \rightarrow \alpha X \beta$ where $\epsilon \in \text{First}(\beta)$
 - $\text{Follow}(A) \subseteq \text{Follow}(X)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

$$E \rightarrow TX$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- $\$ \in \text{Follow}(E)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$
- $\text{Follow}(X) \subseteq \text{Follow}(E)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$ because $\varepsilon \in \text{First}(X)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$
- $\text{Follow}(X) \subseteq \text{Follow}(E)$
- $\text{Follow}(Y) \subseteq \text{Follow}(T)$

Computing the Follow Sets (for the Non-Terminals)

- Recall the grammar

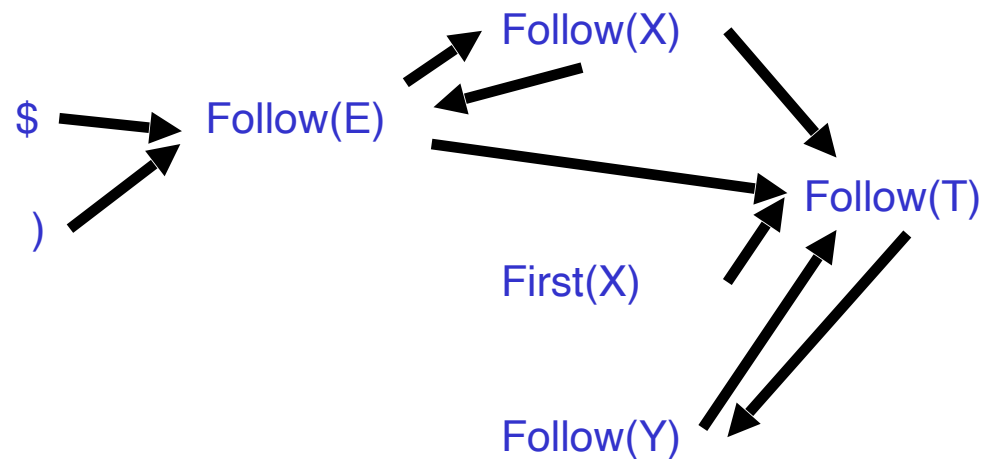
$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

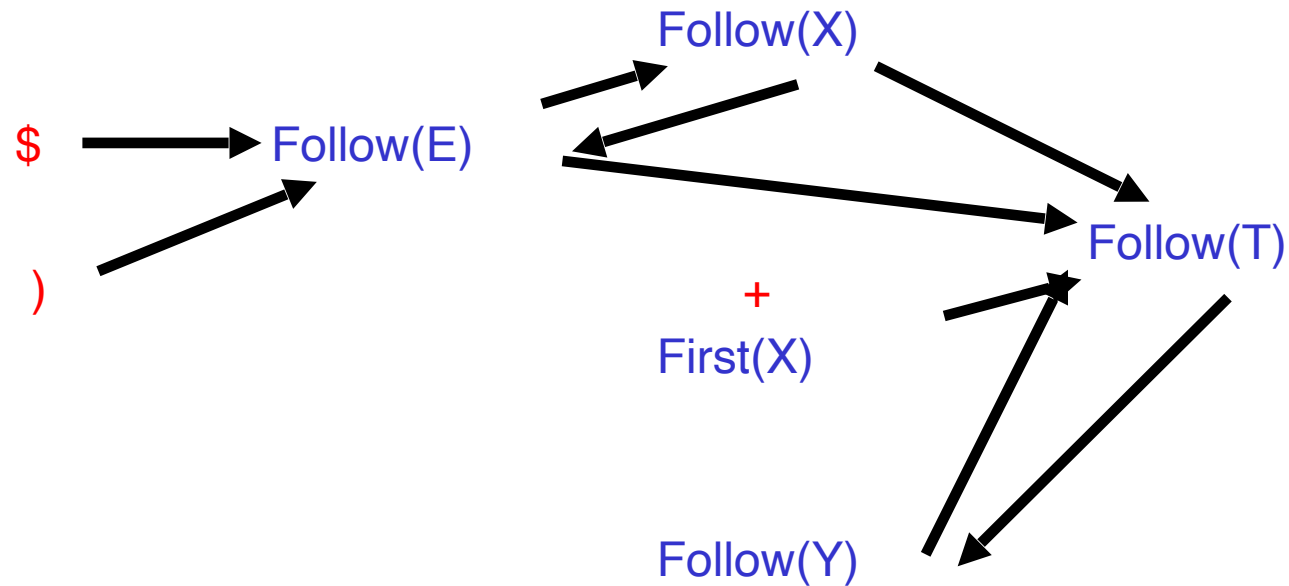
$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

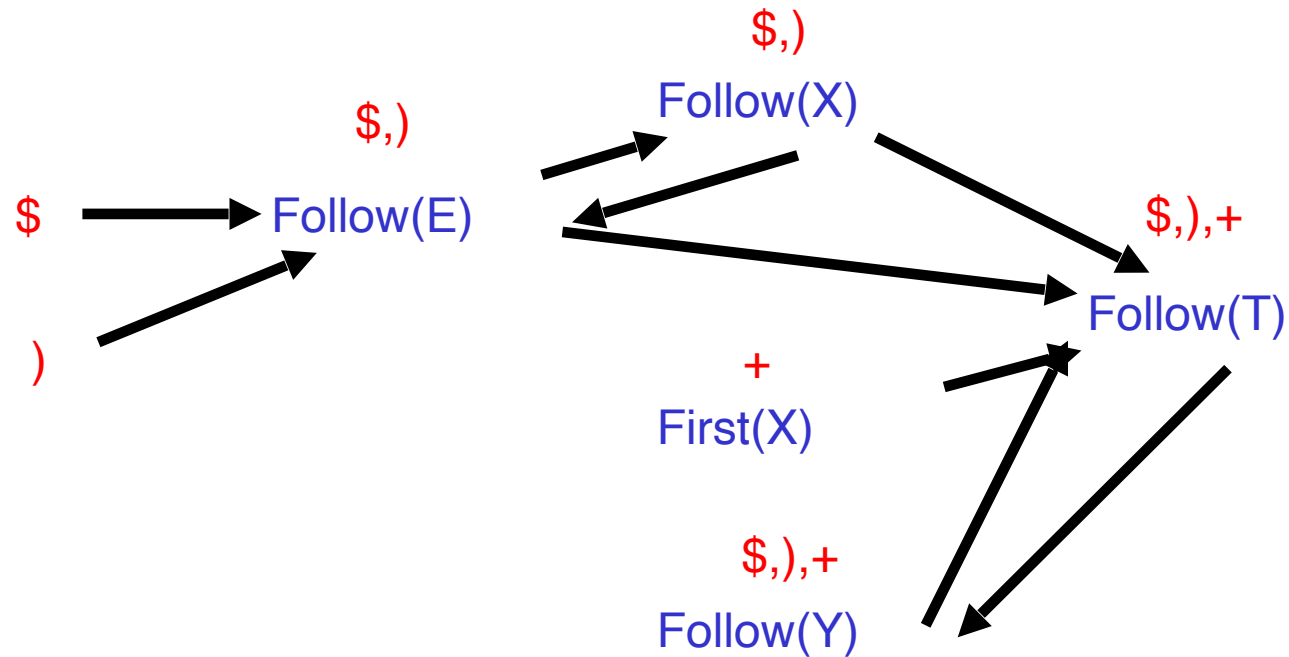
- $\$ \in \text{Follow}(E)$
- $\text{First}(X) \subseteq \text{Follow}(T)$
- $\text{Follow}(E) \subseteq \text{Follow}(X)$
- $\text{Follow}(E) \subseteq \text{Follow}(T)$
- $) \in \text{Follow}(E)$
- $\text{Follow}(T) \subseteq \text{Follow}(Y)$
- $\text{Follow}(X) \subseteq \text{Follow}(E)$
- $\text{Follow}(Y) \subseteq \text{Follow}(T)$



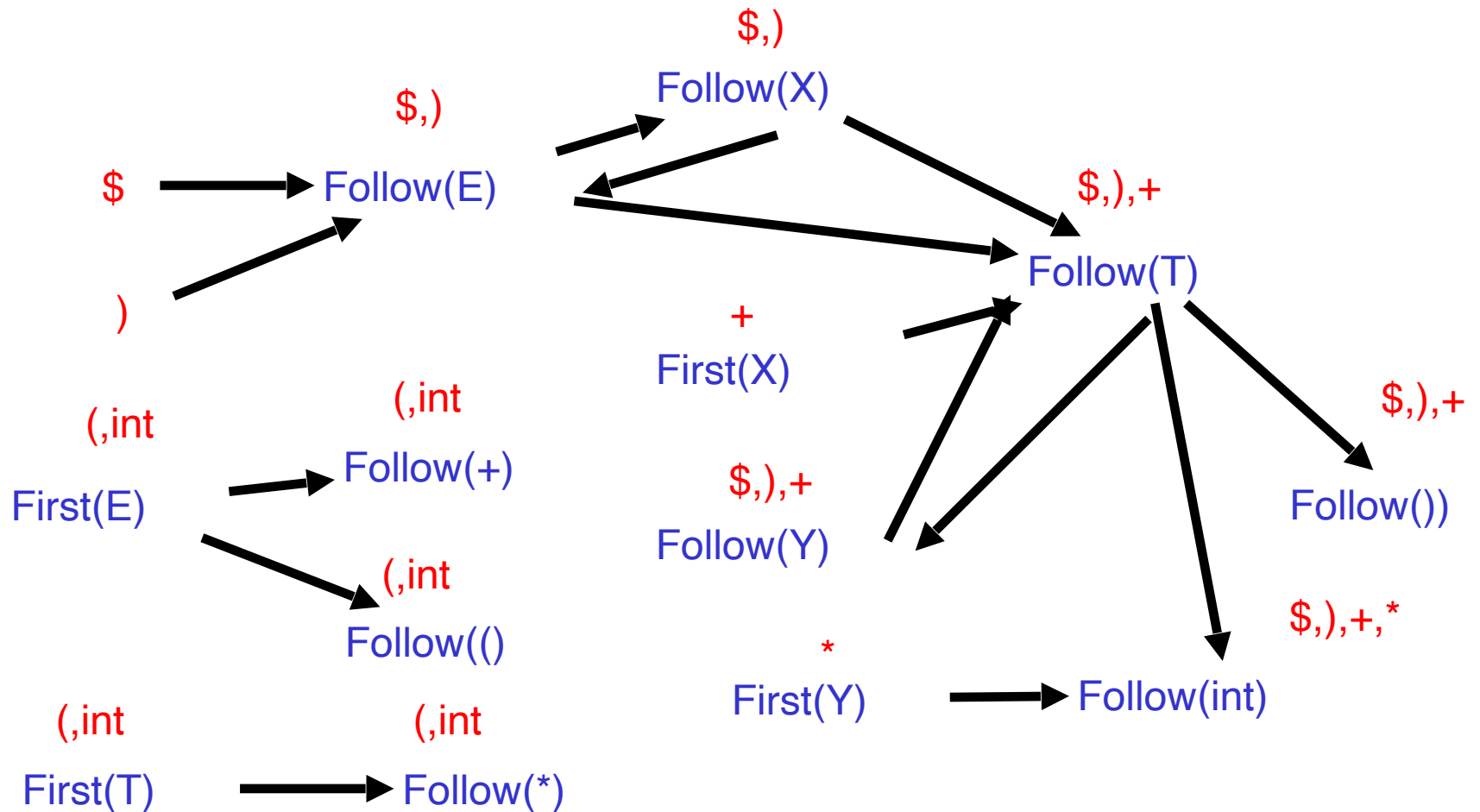
Computing the Follow Sets (for the Non-Terminals)



Computing the Follow Sets (for the Non-Terminals)



Computing the Follow Sets (for all symbols)



Follow Sets: Example

- Recall the grammar

$$E \rightarrow T X$$

$$T \rightarrow (E) \mid \text{int } Y$$

$$X \rightarrow + E \mid \varepsilon$$

$$Y \rightarrow * T \mid \varepsilon$$

- Follow sets

$$\text{Follow}(+) = \{\text{int}, (\}$$

$$\text{Follow}(() = \{\text{int}, (\}$$

$$\text{Follow}(X) = \{\$,)\}$$

$$\text{Follow}() = \{+,), \$\}$$

$$\text{Follow}(\text{int}) = \{*, +,), \$\}$$

$$\text{Follow}(*) = \{\text{int}, (\}$$

$$\text{Follow}(E) = \{\$,)\}$$

$$\text{Follow}(T) = \{\$, +,)\}$$

$$\text{Follow}(Y) = \{\$, +,)\}$$

Constructing LL(1) Parsing Tables

- Construct a parsing table T for CFG G
- For each production $A \rightarrow \alpha$ in G do:
 - For each terminal $t \in \text{First}(\alpha)$ do
 - $T[A, t] = \alpha$
 - If $\epsilon \in \text{First}(\alpha)$, then for each $t \in \text{Follow}(A)$ do
 - $T[A, t] = \epsilon$
 - If $\epsilon \in \text{First}(\alpha)$ and $\$ \in \text{Follow}(A)$ do
 - $T[A, \$] = \epsilon$

Notes on LL(1) Parsing Tables

- If any entry is multiply defined then G is not LL(1)
 - If G is ambiguous
 - If G is left recursive
 - If G is not left-factored
 - And in other cases as well
- Most programming language CFGs are not LL(1)